



Università
di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e
Informatica

Advanced Programming Languages

C++: intro

Docente:

Marco Grassia

marco.grassia@unict.it

A.A. 2025/2026

Your first (??) C++ program

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello, World!" << std::endl;  
  
    std::cout << "Welcome" << "to C++";  
  
    return 0;  
}
```

loads a *header* file containing function and class definitions

here, we load the C++'s standard I/O library
the functions and definitions included belong to a *namespace* called *std* ("STandarD")

- `main()` is the entry point of the program
- statements end with `;`

- `cout` is the *object* that writes to the `stdout` device, i.e., the console window
- It is part of the C++ standard library "iostream"
- `<<` is the C++ *insertion operator*, used to pass characters from the right to the object on the left
- `endl` is the C++ newline character (but also flushes the stream, prefer `\n` in the middle of sentences)
- like C, the program must return an integer (implicitly 0 if omitted) signaling the exit error code.
- Any code `!= 0` means error

Basic syntax

- **Similar to C, Java, C#** in structure
- **Curly braces {}** define a **block of code**
- **Statements** end with a **semicolon ;**
- **Whitespace, tabs, and newlines** have **no syntactic meaning** (unlike Python)
 - Use **indentation** for readability

Structure of a C++ program

- A C++ program consists of **one or more translation units** (source files + included headers – after the pre-processing step)
 - .cpp for source, .hpp for headers
 - **Header files are not compiled separately**; they are included during preprocessing
- **Exactly one** translation unit defines **main**, the **program entry point**
- Code is organized using **namespaces, functions, classes/structs**, and optionally **modules** (C++20)

Tokens (lexical elements)

- A **token** is the smallest meaningful element in the source code.
- C++ tokens include:
 - **Identifiers**: names for variables, functions, classes, etc.
 - **Keywords**: reserved words like int, class, return
 - **Literals**: fixed values written directly in code:
 - **Operators**: + - * / && || :: etc.
 - **Punctuation**: {} () ; , [] <>

Identifiers

- An **identifier** (or **symbol**) is a **sequence of characters in the code that identifies**: a **variable**, a **function**, a **type** or a **label**
- **An identifier is NOT a string!**
 - A string is a constant datum (a literal), and is indicated by double quotation marks (e.g., "hello world")
 - An identifier is part of the code, and does not include quotation marks

Identifiers

- Character sequences such that:
`<Identifier> ::= <Letter> {<Letter> | <Digit>}`
- **At least one initial letter, possibly followed by other alternating letters and/or numeric digits**
 - The **letters** are all **Latin characters** (a-z, A-Z) **plus** the underscore “_”
 - Note that names are **case-sensitive** : myVar ≠ myvar
 - By **convention**, an **initial underscore indicates** “private” names, and **should be avoided in user-code** in C++ as a best practice
 - **Numeric digits at the beginning of the name are not allowed**
- **Length can be 1+ chars**

Keywords in C

- **Keywords** are identifiers with a special meaning, reserved for language features. They **cannot be used as** variable names, function names, or any other **identifiers**

- **Common C++ keywords** (ISO standard):

*alignas alignof asm auto bool break case catch
char class const constexpr consteval constinit
continue decltype default delete do double else
enum explicit export extern false float for friend
goto if inline int long mutable namespace new
noexcept operator private protected public
register reinterpret_cast requires return short
signed sizeof static_assert static_assert struct switch
template this thread_local throw true try typedef
typeid typename union unsigned using virtual void
volatile wchar_t while*

Keywords

alignas	alignof	asm	auto	bool	break	case
catch	char		class	const	constexpr	constexpr
constexpr	continue		decltype	default	delete	do
double	else		enum	explicit	export	extern
false	float		for	friend	goto	if
inline	int		long	mutable	namespace	new
noexcept	operator		private	protected	public	register
reinterpret_cast	requires		return	short	signed	sizeof
static	static_assert		struct	switch	template	this
thread_local	throw		true	try	typedef	typeid
typename	union		unsigned	using	virtual	void
volatile	wchar_t		while			

Literals (literal)

- ***Literal*** values are **constants** (immutable values) — as opposed to variables — **that are written "literally"** within the program code
- Examples:

Integers

```
4  
1525
```

Strings (double quotes)

```
"Hello World"
```

Hexadecimal numbers,
represented by an initial *0x*

```
0xFF  
0xA1B
```

Real numbers

```
4.3  
.05  
0.05  
3.1E+2  
3.14e2
```

Characters (single quotes)

```
'a'  
'b'
```

Literals (literal)

- ***Literal*** values are **constants** (immutable values) — as opposed to variables — **that are written “literally”** within the program code
- Examples:

Boolean
values

```
true  
false
```

Binary numbers (C++14)
represented by an initial 0b

```
0b1010  
0b1111
```

Octal numbers
represented by an initial 0

```
0123  
045627
```

Null pointers

```
nullptr
```

C++ type system:

Strongly typed & statically typed

- **Strongly typed & statically typed:** every object has a type that **never changes**
- As in most programming languages, in C/C++ **each variable** (and constant) **is associated with a data type** that defines the size of the associated memory region, its format, the possible values, the applicable operators, ...
- **The association** of a **variable name** to a data **type is static and never changes during program execution**
- The **compiler performs static checks** (at the time of compilation) **on the correctness of the program**
- The **type of literals is automatically inferred**

Variables

- A **variable** is an **abstraction of a memory cell**
- Each variable:
 - Has a **name** (symbolic identifier);
 - The name is associated (at runtime) with a **memory address** (assigned at each program execution)
 - Has a **type**, which determines:
 - The set of values it can hold
 - The operations allowed on it
 - The value of the variable can be **initialized, read, and modified** during program execution

Declaring Variables

- Syntax
 <type> <identifier>;
- You can group declarations of multiple variables of the same type into a comma-separated list
 <type> <identifier_1>, < identifier _2>, ... < identifier_N>;
- Examples:
 int x; /* x must denote an integer value */
 float y; /* y must denote a real value */
 char ch; /* ch must denote a character */
 int a, b, c;
 float z, w;

Assignment statement

- The **assignment statement** is used to assign the value of an expression to a variable (not a constant)
- Denoted by the symbol **=** (different from the equality relational operator which is denoted by the symbol **==**). Syntax:

`<identifier> = <expr> ;`

Expression
(denoting a
value)

Variable
Name

- Examples:
a = 34;
b = 34 + a*4;

Assignment statement

- Executing an assignment statement first evaluates the expression (to the right of the assignment)

- Ex:

```
c = 3;           /* now c is 3 */  
d = (c+7)/2 - 4; /* now d is 1 */  
d = (d-c)*2;     /* now d is worth -4 */
```

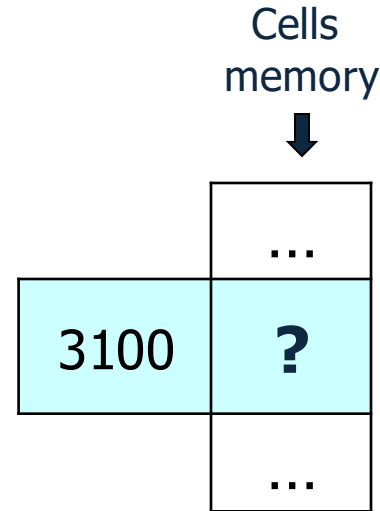
- After that, the resulting value is inserted into the memory location relative to the variable (located to the left of the assignment)

Assignment Statement

Example

int N;

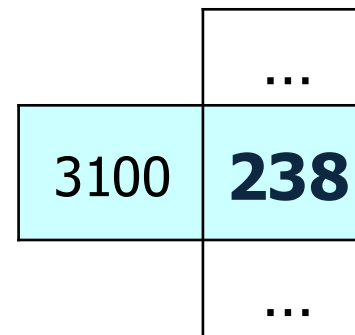
<i>symbol</i>	<i>address</i>
N	3100



Executing a **declaration** allocates memory space equivalent to that required to hold data of the specified type

N = 238;

<i>symbol</i>	<i>address</i>
N	3100



Performing an **assignment** causes the value to be inserted to the right of the = symbol in the variable space

l-value and *r-value*

- A **l-value** (“locator value” or “left” value) refers to an **object that occupies an identifiable location in memory**
 - It can appear on the **left-hand side** of an assignment
- A **r-value** (“read value”) represents a **temporary or computed value**, a data value that does **not necessarily have a persistent memory address**
 - It can appear on the **right-hand side** of an assignment
 - **Sometimes used to describe the value of an expression** and to distinguish it from an l-value
- **All l-values are r-values but not all r-values are l-values**

Initializing Variables

- At the same time as the *definition*, you can *specify an initial value* for a variable

- **Initializing (first assigning)** a variable:

<type> <identifier> = <expr> ;

<type> <identifier1> = <expr_1>, ..., <identifier_N> = <expr_N>;

- Example

int x = 32;

double speed = 124.6;

double z = 16, y = 32;

Compact assignment operators

- C introduces a *particular form of assignment* that **also includes an operation**:

$\langle identifier \rangle \textbf{OP} = \langle expression \rangle$

is **almost equivalent** to:

$\langle identifier \rangle = \langle identifier \rangle \textbf{OP} \langle expression \rangle$

where **OP** indicates an operator (for example: **+**, **-**, *****, **/**, **%**,...)

Compact assignment operators

$a = a + b;$ $\rightarrow$ $a += b;$

$a = a - b;$ $\rightarrow$ $a -= b;$

$a = a * b;$ $\rightarrow$ $a *= b;$

$a = a / b;$ $\rightarrow$ $a /= b;$

$a = a \% b;$ $\rightarrow$ $a \% = b;$

Compact assignment operators

- Pay attention to the priority of operators when using compact operators

- For example:

$x *= y + 3;$

It is equivalent to:

$x = x * (y + 3);$

It is not equal to:

$x = x * y + 3;$

<block> ::= {

[<Declarations and Definitions>]

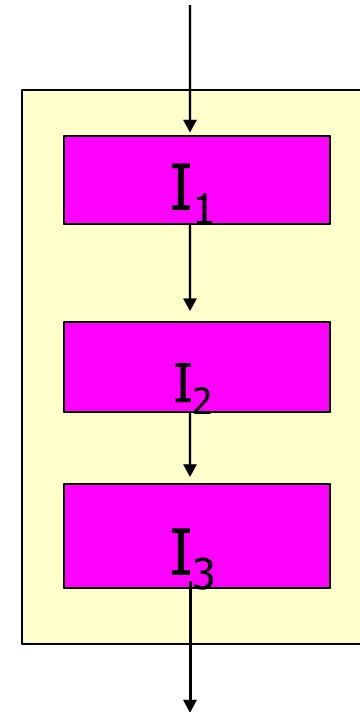
{ <instruction> }

Flowchart

C

Code blocks

- A **block** of code is **delimited in curly brackets**
- Blocks are **compound instructions** and **can be nested** together!
- **After a block** (i.e., after the “}”) you **don't need a semicolon**
- **By convention**, we use **tabular** to the **right** the instructions belonging to that block of code
- **Variables** can be declared **anywhere inside a block** (after C++98)



```
{  
  I1;  
  I2;  
  I3;  
}
```

```
{  
  {  
    I1;  
  }  
  I2;  
  I3;  
}
```

Comments

Comments are **text that is ignored by the compiler, used to describe the code and make it more understandable**

Two ways in C:

- In-line: anything that follows two slashes (//) is a comment
- Block comment: everything between /* and */ is ignored. They can also span multiple lines but cannot be nested

```
// a one line comment
```

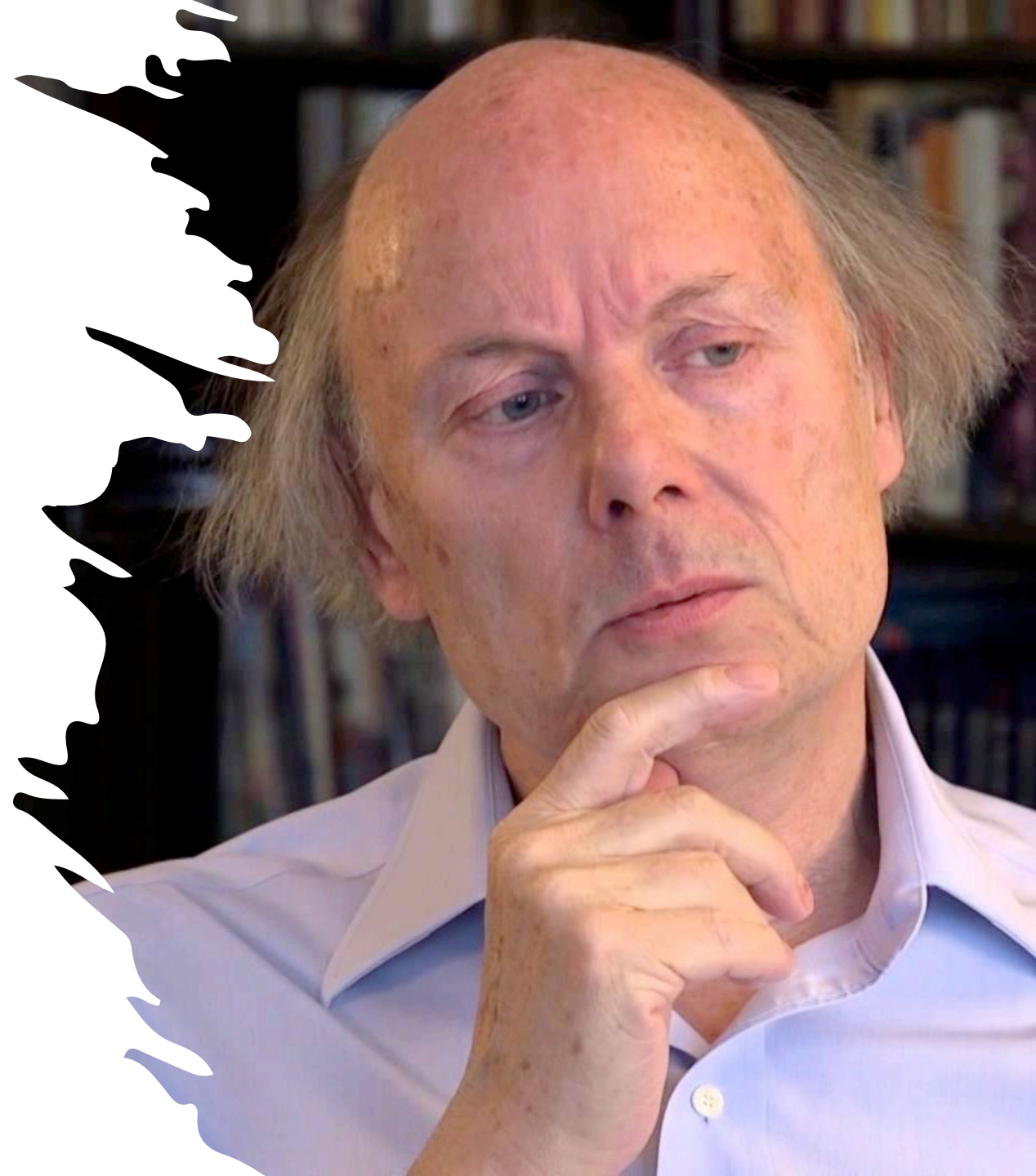
```
/* this is also one line */
```

```
/* this is a longer,  
 * multi-line comment  
 */
```


*“If you think it's simple, then
you have misunderstood the
problem.”*

– Bjarne Stroustrup

(developer of the C++ language)



Introduction to I/O in C++

- Like C, **C++ does not have built-in input/output operations**
- Instead, the **Standard Library** provides objects for:
 - **Reading input** (e.g., from keyboard or files)
 - **Writing output** (e.g., to console or files)
- Main headers:
 - `<iostream>` → console I/O
 - `<fstream>` → file I/O
- Uses **stream objects**

Streams

- The **stream** is the core abstraction:
 - A stream **represents a source or destination of bytes**
 - Think of it as a **smart file** that knows how to handle data flow to any device
 - It interacts with the devices through the OS, which treats them as **extended files**
- **Standard Streams**
 - **std::cin** → standard input (keyboard)
 - **std::cout** → standard output (console)
 - **std::cerr** → error output (unbuffered)
 - **std::clog** → error output (buffered)

Basic output with std::cout

- << is the **insertion operator**, and it's **overloaded for many types**
- `std::endl` = newline + **flush**, it can hurt performance
- You can switch format bases with flags (`std::hex`, `std::dec`, ...).

```
#include <iostream>
```

```
int main() {  
    std::cout << "Hello, " << 42 << '\n';  
    // chain different types  
  
    std::cout << std::boolalpha << (3 < 5) << '\n';  
    // prints: true  
  
    // Avoid std::endl unless you need a flush:  
    std::cout << "line\n"; // faster than std::endl  
}
```

Basic input with *std::cin*

- `>>` is the **extraction operator**, and it's **overloaded for many types**
- Use the **operator ()**

```
#include <iostream>
```

```
int main() {  
    int x;  
    std::cin >> x;  
    std::cout << "You entered: " << x << std::endl;  
}
```

Conditional statements

if, if-else, switch

Conditional statements in C

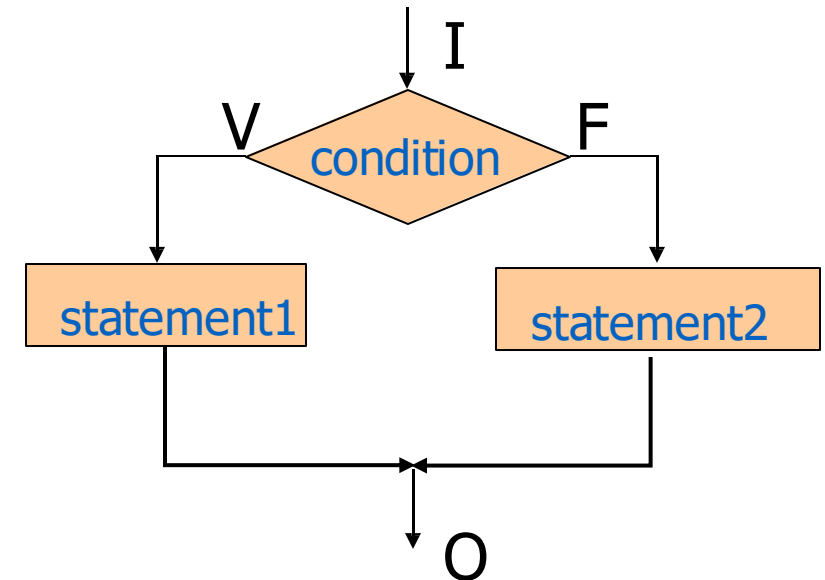
- **Two types of conditional statements:**
 - Instruction of **choice (simple)**
 - **if** (one way)
 - **if-else** (two-way)
 - Multiple choice **instruction *switch*, that enhances expressiveness**, and does not violate the constraints of structured programming
- There is also **a conditional expression:**
 - **Ternary conditional expression**

Simple choice statement: if[-else]

```
<conditional_statement> ::= if ( <expression> ) <statement>  
                             |  
                             if ( <expression> ) <statement> else <statement>
```

- The **condition** is a **logical expression** that is **evaluated at the time of execution of the if statement**
- In C, **any non-zero value is considered as “true”**
- **The statement can be a block** of code, executed only if the condition is true, **and the else is optional** (statement executed only if the condition is false)
- Execution then continues with the statement following the if-else

```
if ( condition )  
    instruction 1;  
else  
    instruction 2;
```



if then, else if, else

- The else also supports other nested ifs, forming chains of ifs – else if – else

```
if (condition) {  
    /* code to run if condition is true */  
}  
else if (condition2) {  
    /* code to run if condition2 is true */  
}  
else if (condition3) {  
    /* code to run if condition3 is true */  
}  
else {  
    /* run some other code instead */  
}
```

Nesting *if* and *else* without brackets

- What is printed in both cases?

```
if (true) if (false) printf("A");  
    else printf("B");
```

'B'

```
if (false) if (true) printf("A");  
    else printf("B");
```

Nothing, we never enter the code block of the external if!

The else is **always referred** to the last if

Please note: else is always associated with the innermost if, blocks are used to change the semantics

true and *false* in C and C++

- Traditionally, in **C**, **boolean values are** simulated using **integer (*int*) data types**
 - **0** represents ***false***
 - **Any value other than 0** represents ***true***
 - There's a `stdbool` library
- C++98 introduces a native `bool` type with *true* and *false* values as part of the language

```
int x = 0;
if (x) {
    This block will not be executed,
    // because x is false (0).
}

int y = 42;
if (y) {
    This block will be executed,
    // because y is true (different from 0).
}
```

Relational operators

- Relational operators **compare two values or expressions and return a Boolean result**

Relation	C
Equality	==
Diversity	!=
Greater than	>
Less than	<
Greater than or equal to	>=
Less than or equal to	<=

Logical operators

- **Logical operators perform logical operations on Boolean values or conditional expressions**

Logical connective	Operator	C
NOT (negation)	Unary	!
AND	binary	&&
OR	binary	

Multiple choice statement: switch

- A ***switch*** statement **evaluates an expression and attempts to compare its value with various constants**, made explicit by means of ***case labels***
- **If a match is found, the matching code block is executed**
- **Otherwise, the *default* clause (optional) is executed**
- **The *break* statement stops the switch from running at that point.**
If it is omitted, subsequent instructions are also executed (up to the first break)!

```
switch (condition) {  
  Case 1:  
    /* code to run if  
       condition == 1 */  
                                break;  
                                case 2:  
    /* code to run if  
       condition == 2 */  
                                break;  
  default:  
    /* run some other  
       code instead */  
                                break;  
}
```

Multiple choice statement: switch

- Syntax:

```
<switch> ::= switch ( <expression> ) { { <case> } }  
<case> ::= case <constant_value> : < instruction> |  
          default" : < instruction>
```

- The **value of the** selector expression (an **integral!**) is **compared to labels** (**constants of the same type** as the selector)
- **Execution continues** from the **corresponding** branch (*case*) **if it exists**
- **If the expression is not present** in the cases, the **default block statement is executed**, which is still optional

Example of switches and notes

<statement> denotes a sequence of instructions, you don't need to specify a block

The various branches (case) are not mutually exclusive: once a branch is taken, the execution generally continues with the following branches

To have mutually exclusive branches, you must explicitly force the exit using the *break* statement

```
printf("Please enter a number (1-3): ");  
scanf("%d", &choice);
```

```
switch (choice) {  
    case 1:  
        printf("You chose the first option.\n");  
        break;  
    case 2:  
        printf("You chose the second option.\n");  
        break;  
    case 3:  
        printf("You chose the third option.\n");  
        break;  
    default:  
        printf("Invalid choice.\n");  
}
```


Multiple choice statement: switch

```
switch (condition) {  
    case 1:  
        /* code to run if condition == 1 */  
        break;  
    case 2:  
        /* code to run if condition == 2 */  
        break;  
    default:  
        /* run some other code instead */  
        break;  
}
```

```
switch (condition) {  
    case 1:  
        /* code to run if condition == 1 */  
  
    case 2:  
        /* code to run if condition == 1 or 2 */  
        break;  
    default:  
        /* run some other code instead */  
        break;  
}
```

Multiple choice instruction: switch – pros and cons

- The switch instruction **avoids a (long) series of ifs**
- **However:**
 - It can **only be used with integer** expressions and **labels** (*int*, *char*, enum, and all variants)
 - and, therefore, it cannot be used **with real numbers** (float, double) **or with structured types** (strings, vectors, structures, ...)

Conditional (ternary) operator

```
(condition) ? expression_if_true : expression_if_false
```

- The conditional ternary operator selects between two values depending on a condition

```
(condition) ? val1 : val2
```

- If *condition* is *true*, it resolves with *value val1*, otherwise *val2*
- ```
int main() {
 bool x = true;
 printf("%s", x ? "true" : "false");
}
```
- It's often used for assignments

```
int x = (condition) ? 1 : 2;
```

Equivalent to writing:

```
int x;
if (condition) x = 1;
else x = 2
```

# Iteration Instructions

while, do-while, for

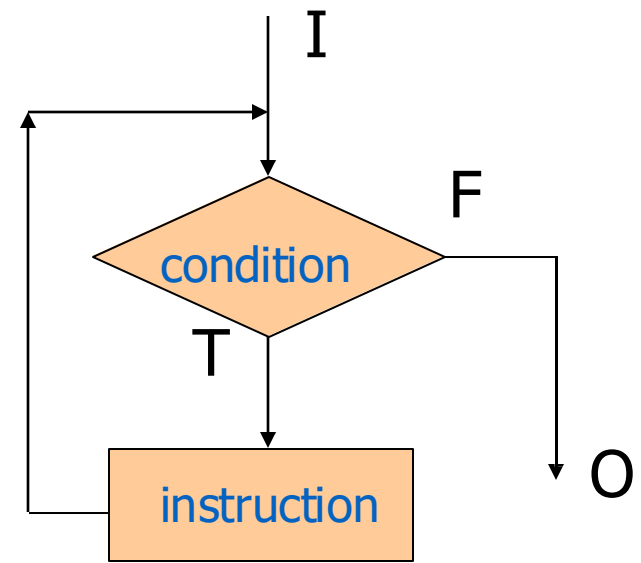
# Iteration Instructions in C

- Iteration statements **provide** control **structures to repeat** a certain **statement during the occurrence of a certain condition**
  - **The instruction** executed **can be a block!**
- **By the Jacopini-Böhm Theorem, a single iterative** control structure **is sufficient** to compute every computable function...
- ... but **to improve the expressiveness** of the language, **in C**, the following **are available**:
  - **while** (<condition>) <statement>
  - **do** < statement> **while** (<condition>);
  - **for** (...) <statement >

# Looping: *while*

```
while(condition) statement;
```

- ***while* repeats a block of code... while... a given condition is true**
  - The *statement* (instruction) can be a block of code!
- ***How does it work?***
  1. The ***condition*** is evaluated: if it is **false** (resolves to 0), the loop is broken and the statement is continued after the while
  2. The ***statement*** is executed
  3. Back to 1



```
while(x < 10) {
 printf("%d\n", x);
 x++;
}
```

# Looping: *while*

```
while(condition) statement;
```

- If the **condition** is already **false** initially, the **iteration** is **never** executed
- Sooner or later, **directly** or **indirectly**, the **statement** must **change** the **condition**: **otherwise**, we get an **infinite loop**
- For this reason, **< statement >** is usually a **block**, which **contains at least one statement** in which you **modify** some **variable** that **appears** in the **condition**

```
while(x < 10) {
 printf("%d\n", x);
 x++;
}
```

Infinite loop:  
x is never changed,  
and the condition will result  
always true  
(if true from the start)

```
while(x < 10) {
 printf("%d\n", x);
}
```

# Example #1

- Calculation of the average of N marks
- Each vote must be  $\geq 0$  and  $\leq 30$ !

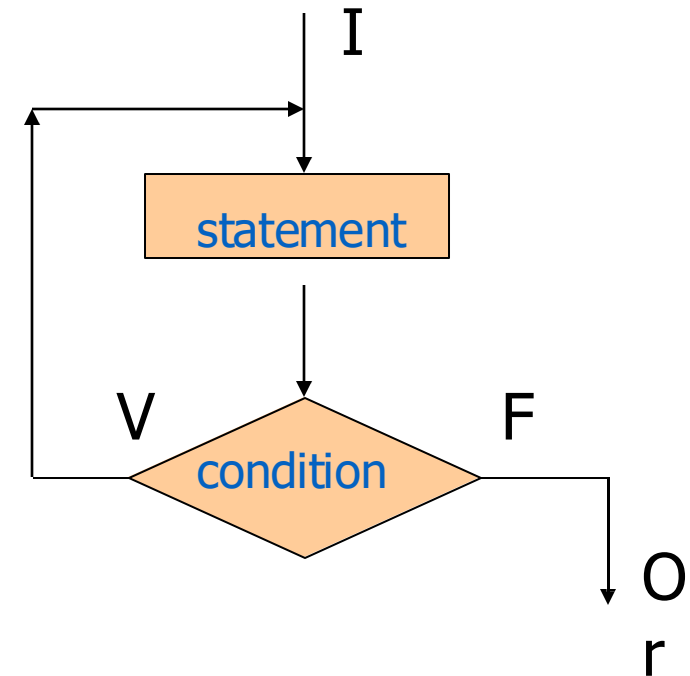


# Looping: *do... while*

```
do statement while (condition);
```

- *do... while* **executes** (do) a **statement (or block) and repeats it** *while* (while) a **condition** is verified
- How does that work?
  1. The *statement* statement is executed
  2. The *condition expression* is evaluated, if *false* the loop is terminated
  3. Back to 1

```
do {
 printf("%d\n", x);
 x++;
} while (x < 10);
```



# goto and labels

- **goto** transfers control unconditionally to a **label** within the same function (before or after)
- **Dangerous and should be avoided**
  - Leads to **unstructured control flow**, making code harder to read and maintain
  - Breaks the principle of **structured programming**
  - Commonly associated with **spaghetti code**:
    - Code with tangled jumps and unclear logic
    - Difficult to debug and extend

```
goto label_name;
// ...
label_name:
// ...
```

# goto restrictions

- Cannot jump into a scope that introduces new variables or skip variable declaration and the labeled code uses it
- Cannot cross object initialization that would break RAI semantics

```
goto label_name2;
// ...
```

```
label_name:
int a = 10;
```

```
label_name2:
a = 20;
```

error: cannot jump from  
this goto statement to its  
label

note: jump bypasses  
variable initialization

# goto best practices

- Avoid goto except for:
  - **Error handling** in low-level code (e.g., cleanup in C-style APIs)
  - Situations where structured alternatives (loops, exceptions) are not viable
- Prefer:
  - **Loops** (for, while)
  - **Functions**
  - **Exceptions** for error handling in C++

```
goto label_name2;
// ...
```

```
label_name:
int a = 10;
```

```
label_name2:
a = 20;
```

error: cannot jump from  
this goto statement to its  
label

note: jump bypasses  
variable initialization

# Example #2 (Input Validation)

- Reading a positive integer from the keyboard
- If the user provides an invalid number, try again
- Variants:
  - n must be negative or between 10 and 20
  - n must be between 3 and 15

# Example #3

- Calculating the factorial of a number  $N$  (positive!)

# Example #4

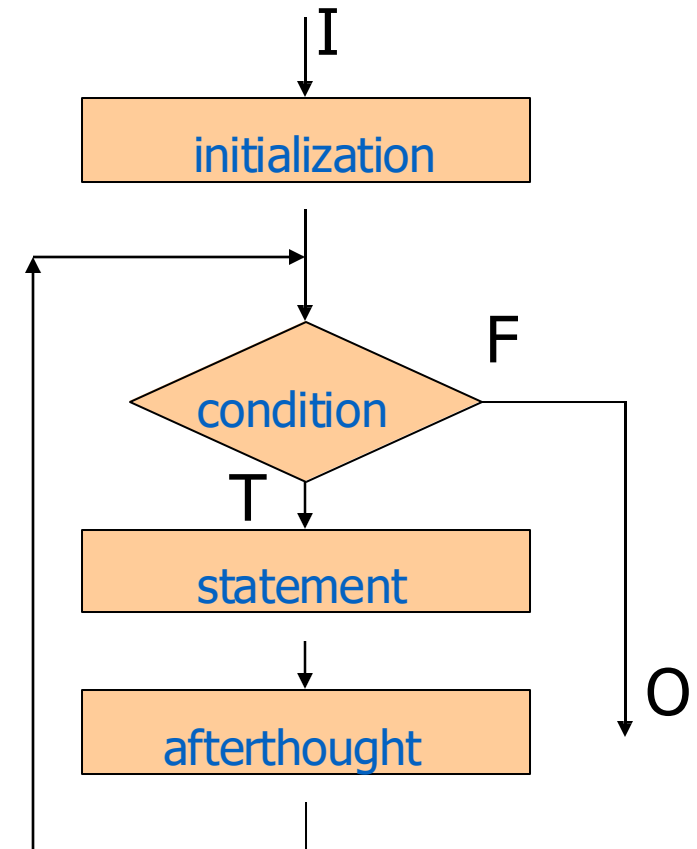
- Read from input a sequence of positive integers ending with a value of -1

# Looping: *for*

```
for (initialization; condition; afterthought) statement;
```

- *for* iterates until *condition* is *true*. Specifically:
  1. The *initialization expression* is executed, if present
  2. The *condition* condition is verified. If false, the loop ends
  3. The statement (or block) *is* executed
  4. At this point, if present, the *afterthought expression* *is* executed
  5. Back to 2

```
for (int i = 0; i < 10; i++) {
 printf("%d\n", i);
}
```





# *for*

- **Used when the number of iterations is known** or can be determined
- It **evolves from *while* to reduce common errors**:
  - Missing initialization
  - Missing update (risk of infinite loop)
- **Components**:
  - **Initialization**: executed once before the loop starts
  - **Condition**: checked before each iteration
  - **Update**: executed after each iteration

# Iterative statement *for*

- **Used when the number of iterations is known** or can be determined
- It **evolves from *while* to reduce common errors**:
  - Missing initialization
  - Missing update (risk of infinite loop)
- The **update expression** can be **any valid expression**, not just *i++* or *i--*

*// logarithmic growth*

```
for (int i = 1; i < n; i *= 2) {
 std::cout << i << '\n';
}
```

*// halving*

```
for (double x = 1.0; x > 0.001; x /= 2) {
 process(x);
}
```

*// step increment or decrement*

```
for (int i = 0; i < n; i += 10) {
 std::cout << i << '\n';
}
```

# Control the loop: *break* and *continue*

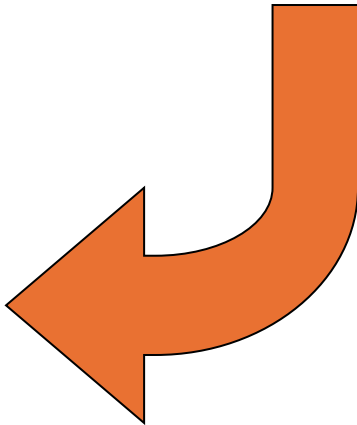
- ***break*... breaks (ends) the loop immediately**
  - It works with *for*, *while*, *do-while*... and switches
- ***Continue* continues the loop by jumping directly to the next iteration** (that is, skipping the following statements in the code block)
  - In a *while* loop, **jump directly to the condition evaluation**
  - In a *for* loop, **jump directly to the after-thought expression** (usually the **increment**)

```
while(true) {
 if (condition) {
 break;
 }
}
```

```
for (int i = 0; i < 10; i++) {
 if (condition) {
 continue;
 }
 printf("%d\n", i);
}
```

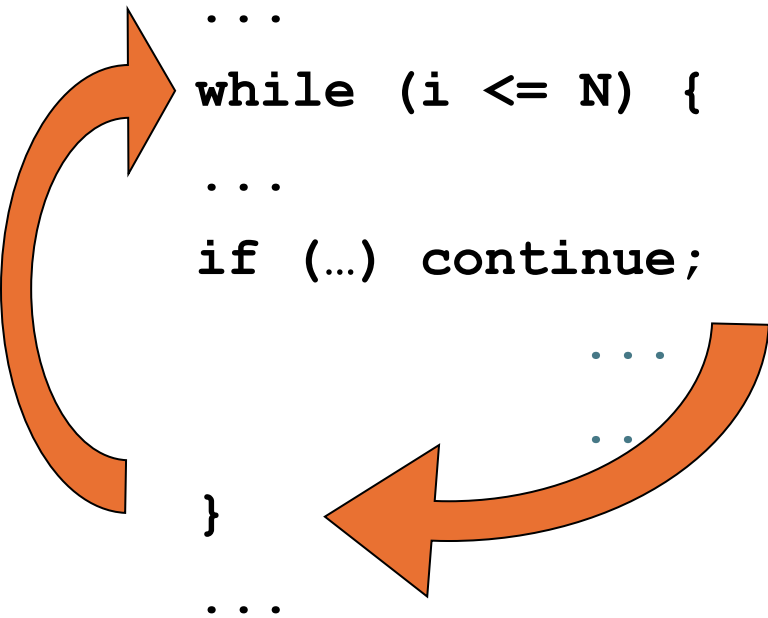
# Break Statement

```
...
while (i <= N) {
 ...
 if (...) break;
 ...
 ...
}
...
```



When the break is performed,  
control is transferred to the **First  
Subsequent Instruction** to the  
block where the break is located

# *continue*



When the `continue` is performed, control is transferred to the **end of the block** causing a **reassessment of the condition that controls the cycle**

**The instructions** skipped between are the ones between the *continue* and the *parenthesis }* that closes the block